



UNIVERSITY OF PERADENIYA
DEPARTMENT OF STATISTICS AND COMPUTER SCIENCE
END SEMESTER EXAMINATION - SEMESTER II (2022/2023)
CSC1051 - PROGRAMMING LABORATORY II

Answer **ALL** Questions

Time Allowed: 3 Hours

Create a folder named **CSC1051_End_S21XXX** in your H drive. Create a new Java project named **S21XXX**. Create two packages named **Q1** and **Q2** inside the created Java project. Implement the answer for question 1 and question 2 inside **Q1** and **Q2** packages respectively. Save all your work inside the **CSC1051_End_S21XXX** folder created in your H drive. Only the files stored in this folder will be considered grading.

Question 1 [70 Marks]

You are tasked with developing a software system for an **Automated Bee Conservation System** for monitoring bees, surveillance drones and their pollination impact on crops.

You are required to create constructors for each class with necessary parameters.
Marks will be allocated for following OOP concepts and best programming practices.

- a. Create an interface named **Monitorable** with a method **String generateReport()** to generate reports.
- b. Create an **abstract** class **PollinationMonitor** that implements the **Monitorable** interface.

Attributes

- **String monitorId**: Unique identifier for the pollination monitor.
- **String monitorType**: Type of pollination monitor (e.g., Bee, Drone).

Methods

- Create an **abstract** method **String generateReport()** that returns a report based on the pollination monitor type.
- c. Create a class **BeeSpecies** which is a sub/child class of the **PollinationMonitor** class.

Attributes:

- **String speciesName**: The name of the bee species.
- **String region**: The region where the bee species is primarily found.
- **boolean isEndangered**: Whether the species is endangered or not.
- **double pollinationEfficiency**: Efficiency in pollination.

Methods:

- Create a method named **void updatePollinationEfficiency(double newEfficiency)** to update the pollination efficiency with the given new Efficiency.
- Implement method **void markAsEndangered()** to mark a bee species as endangered and method **void removeFromEndangeredStatus()** to remove the endangered status of the bee species.
- Override **generateReport()** method to return bee species name, region, endangered status, and pollination efficiency.

d. Create a class **PollinationDrone**, which is a sub/child class of the **PollinationMonitor** class.

Attributes:

- **String droneId**: Unique identifier for the drone.
- **boolean isOperational**: Status of the drone (functioning or maintenance).
- **double batteryLevel**: Current battery level of the drone (from 0 to 100%).
- **int tasksCompleted**: Number of tasks the drone has completed.

Methods:

- Implement the method **void performMonitoringTask()** to increment the completed task count by 1 upon task completion.
- Implement the method **void rechargeBattery()** to recharge the drone's battery to 100%.
- Override the method **generateReport()** to return drone's ID, operational status, battery level, and completed tasks.
- Create the method **void optimizeTaskPerformance(int totalTasks)** to calculate the optimal battery usage based on the given number of tasks. Each task consumes **50mWh** of energy and the total battery capacity is **500mWh**. The battery percentage consumed for each task can be calculated using the following formula:

$$\text{Battery percentage consumed} = \left(\frac{\text{Energy consumed in mWh}}{\text{Total battery capacity in mWh}} \right) \times 100$$

Further, this method should,

- Deduct battery percentage based on the energy consumption for each task until the battery percentage goes not less than **15%**.
- Increment the **tasksCompleted** counter appropriately.
- Print the number of tasks performed and the remaining battery level.
- Print a message indicating the remaining number of tasks, if the drone cannot complete all tasks due to insufficient battery.

- e. Create a class **BeeConservationSystem** to track the overall health and pollination efforts of bees.

Attributes:

- **beeSpeciesList:** BeeSpecies array with the size 10 to store all bee species.
- **dronesList:** PollinationDrone array with the size 10 to store all drones.
- **double temperature:** Current temperature in the bee monitoring environment.
- **double humidity:** Humidity level in the conservation area.
- **String pesticideLevel:** Level of pesticides present, ("low"/ "moderate"/"high")

Use the **constructor** to initiate **temperature=26°C**, **humidity=55%**, and **pesticideLevel=Low**

Methods:

- Implement the method **void addBeeSpecies(BeeSpecies species)** to add a bee species to the **beeSpeciesList** array and method **void addDrone(PollinationDrone drone)** to add a drone to the **dronesList** array.
- Implement the **String generateComprehensiveReport()** method to generate detailed reports as shown in **Figure 01**. The generated report should:
 1. Include the details of each bee species.
 2. Include the details of each monitoring drone.
 3. Display the current values of the environmental factors.
- Create **void monitorPollinationHealth()** method to monitor the overall health of bee populations as shown in **Figure 02**:
 1. If the temperature is outside the range of 20°C to 30°C print a warning message.
 2. If the humidity drops below 50% and the pesticideLevel is 'high', print a message.
 3. Identify and print any bee species with a pollination efficiency below 50% with their speciesName.

Expected Output

```
Bee Species Report:
Species: Honeybee, Region: Midwest, Endangered: No, Pollination Efficiency: 65.5%
Species: Bumblebee, Region: North-East, Endangered: Yes, Pollination Efficiency: 48.0%

Drone Activity Report:
Drone ID: D001, Operational Status: Yes, Battery Level: 85%, Tasks Completed: 12
Drone ID: D002, Operational Status: No (Maintenance), Battery Level: 30%, Tasks Completed: 9

Environmental Factors:
Temperature: 26°C
Humidity: 55%
Pesticide Level: Low
```

Figure 01 : Example Comprehensive Report

Warning: Temperature is outside the healthy range (32°C).

Bee Species Alert: Bumblebee has pollination efficiency below 50% (48.0%). Immediate action suggested!

Figure 02 : Example Pollination Report

f. Create a **Main** class to simulate the functioning of the Automated Bee Conservation System.

- Inside the **main** method:

i. Create two **BeeSpecies** with the following details.

Species	Region	Endangered	Pollination Efficiency
Honeybee	Midwest	No	65.5%
Bumblebee	North-East	Yes	48.0%

ii. Create two **PollinationDrones** with the following details

Drone ID	Operational Status	Battery Level	Tasks Completed
D001	Yes	85%	12
D002	No (Maintenance)	30%	8

iii. Print the details of each object using the **generateReport** method.

iv. Call the **optimizeTaskPerformance** method of the **PollinationDrone** class with the **Drone ID: D001** and **Tasks Completed: 15 tasks**.

v. Create a **BeeConservationSystem** instance and add the bees and drones created above to the **BeeConservationSystem** instance using the **add methods**.

vi. Call **generateComprehensiveReport** and **monitorPollinationHealth** methods. (See **Figure 1** and **Figure 2** for example outputs.)

Question 2 [30 Marks]

You are tasked with creating a software application to manage travel itineraries and allow for file writing to save the travel summary.

a. Create a class named **ItineraryItem** with the following attributes:

Attributes:

- **String destination:** The destination of the trip.
- **String date:** The date of the activity (in the format "YYYY-MM-DD").
- **String activity:** The activity planned for the day.
- **double cost:** The cost of the activity.

b. Create a **Main** class to implement the following.

Methods:

- Implement a **static** method named **void writeItineraryToFile()** which accepts an array of **ItineraryItem** objects, and a **fileName** as parameters.

This method writes the formatted itinerary details in **JSON format** in a text file as given in **Figure 3**.

Each itinerary entry should be written in a separate block of attributes. Ensure that the output follows the following JSON rules properly:

- The data should be enclosed in curly braces {}.
- Each itinerary object should be inside an array named "itinerary" (enclosed in box brackets []).
- The key-value pairs should be separated by commas.
- Ensure that the numbers (costs) are written without quotation marks and that the strings (destination, date, activity) are enclosed in double quotes.

The JSON output should be formatted as follows:

```
{
  "itinerary": [
    {
      "destination": "Paris",
      "date": "2024-12-01",
      "activity": "Visit Eiffel Tower",
      "cost": 50.00
    },
    {
      "destination": "Rome",
      "date": "2024-12-05",
      "activity": "Colosseum Tour",
      "cost": 30.00
    },
    ...
  ]
}
```

Figure 3 : Example JSON output

- Implement the **static** method **double calculateTotalCost()**, which accepts an **ItineraryItem** object array, calculates, and returns the total cost of all activities.

1. **In the main method:**

- a. Create four **ItineraryItem** objects with the following details:

Destination	Date	Activity	Cost
Paris	2024-12-01	Visit Eiffel Tower	50.00
Rome	2024-12-05	Colosseum Tour	30.00
New York	2024-12-10	Visit Statue of Liberty	40.00
Tokyo	2024-12-15	Shibuya Crossing Experience	60.00

- b. Add these objects to an array of **ItineraryItem** objects with size 10.
- c. Use the **writeItineraryToFile** method to write the data to a file named ***itinerary_report.txt*** inside the **CSC1051_End_S21XXX** folder located in your **H drive**. (The output should look like **Figure 3**)
- d. Use the **calculateTotalCost** method to calculate and print the total cost of all the activities.

The output should be as follows:

Total cost of all activities: \$180.00

Figure 4 : Example output

_____ ***End of Paper*** _____